

Compte-rendu - Projet RAG Assistant Médicaments

TP : Construire un système RAG avec Python, FAISS, Groq et Streamlit

Sujet choisi Assistant Médicaments	Source BDPM officielle : RCP, substances, compositions	Interface Application Streamlit déployée en ligne	Sécurité Réponses sourcées + refus d'inventer
---------------------------------------	---	--	--

1. Objectif du projet

Dans ce TP, j'ai réalisé un assistant RAG capable de répondre à des questions en langage naturel sur les médicaments. Le système ne répond pas à partir de connaissances générales : il recherche d'abord les passages pertinents dans une base locale, puis le LLM Groq formule une réponse uniquement à partir de ces sources. L'objectif principal est d'obtenir un chatbot médical informatif, sourcé et prudent.

2. Données utilisées et importation

La base utilisée provient de la Base de Données Publique des Médicaments (BDPM). Les fichiers exploités contiennent notamment les RCP, les dénominations des médicaments, les compositions et les substances actives. Une difficulté importante était liée au format des données : fichiers ZIP, fichiers volumineux, encodages différents comme latin-1 ou utf-8-sig, et contenu HTML. Pour résoudre cela, le script d'importation décompresse les fichiers, détecte l'encodage et le séparateur, lit les fichiers par blocs, nettoie les balises HTML avec BeautifulSoup et génère un corpus JSON structuré.

3. Construction de la base vectorielle

Après nettoyage, les documents sont organisés par sections médicales : 4.2 Posologie et mode d'administration, 4.3 Contre-indications, 4.5 Interactions et 4.8 Effets indésirables. Chaque section devient un document avec ses métadonnées : nom du médicament, code CIS, substance active, section et source. Les textes sont ensuite transformés en embeddings avec paraphrase-multilingual-mpnet-base-v2, puis indexés dans FAISS. L'index est sauvegardé dans storage/ afin d'éviter de recalculer les vecteurs à chaque lancement.

4. Fonctionnement du RAG

Lorsqu'un utilisateur pose une question, celle-ci est transformée en vecteur et comparée aux chunks indexés. Le système récupère uniquement les passages les plus pertinents afin de réduire la consommation de tokens. Une recherche hybride a aussi été ajoutée : similarité FAISS + mots-clés, avec filtre strict par nom de médicament et recherche par substance active. Le contexte récupéré est ensuite envoyé au modèle Groq llama-3.1-8b-instant, avec un prompt système qui interdit l'invention d'informations absentes des sources.

5. Interface et fonctionnalités

En plus de la version terminal, j'ai développé une interface Streamlit pour rendre le chatbot utilisable comme une vraie application. Elle permet de poser une question, consulter les sources, afficher les scores de similarité, ouvrir les liens officiels BDPM, voir l'historique de conversation, exporter une réponse en PDF et comparer deux médicaments. L'interface facilite donc l'utilisation du RAG et rend le projet plus présentable qu'une simple exécution en ligne de commande.

6. Sécurité, limites et bilan

Des garde-fous ont été intégrés : seuil de confiance, vérification des termes importants, réduction du contexte envoyé à Groq et mention médicale obligatoire. Chaque réponse rappelle que les informations ne remplacent pas l'avis d'un professionnel de santé. Les principales difficultés ont été le nettoyage du HTML, la taille des fichiers BDPM, les limites de tokens de Groq et le déploiement en ligne sans exposer la clé API. Ces problèmes ont été résolus grâce à la lecture par blocs, aux secrets Streamlit Cloud et à un .gitignore adapté. Au final, le projet respecte le pipeline RAG complet : ingestion, nettoyage, chunking, embeddings, index FAISS, recherche, génération contrôlée, sources et interface utilisateur.

Résumé technique : Python - Pandas - BeautifulSoup - Sentence Transformers - FAISS - Groq - Streamlit - BDPM officielle